

TP1 Introduction to Information Security

Clickjacking: The Button You Didn't Click

September 15, 2026

1 Objective

The objective was to create a local Student Portal, embed it inside a simple game, and show how a transparent iframe can redirect a user's click. All testing was performed locally for educational purposes.

2 First HTML page

The file `hello.html` contains a heading, a paragraph, and a button. The button calls a JavaScript function that displays an alert.

Welcome to My First Website

This is my first HTML page.

Click Me

Figure 1: First HTML page.

3 Student Portal

The file `portal.html` contains a student-name field, a password field, and a Login button. Clicking Login calls `login()` and displays a confirmation alert.

Student Portal

Student name
Password
Login

Figure 2: Local Student Portal.

4 Decoy game

The file `game.html` starts with a score of 0. Each click increments the score and moves the Catch Me button. The horizontal position is selected from 0 to 399 pixels. The vertical position is selected from 100 to 399 pixels so that the button stays below the heading.

```
button.style.left = Math.floor(Math.random() * 400) + "px";
button.style.top = Math.floor(Math.random() * 300 + 100) + "px";
```

Catch Me!

Score: 1

Reveal Hidden Page

Catch Me

Figure 3: Score increment and random button movement.

5 Embedding and hiding the portal

The portal is embedded with an iframe. Setting opacity to 1 makes it fully visible. Lower values make it progressively transparent. At opacity 0, the iframe is invisible but it can still receive clicks.

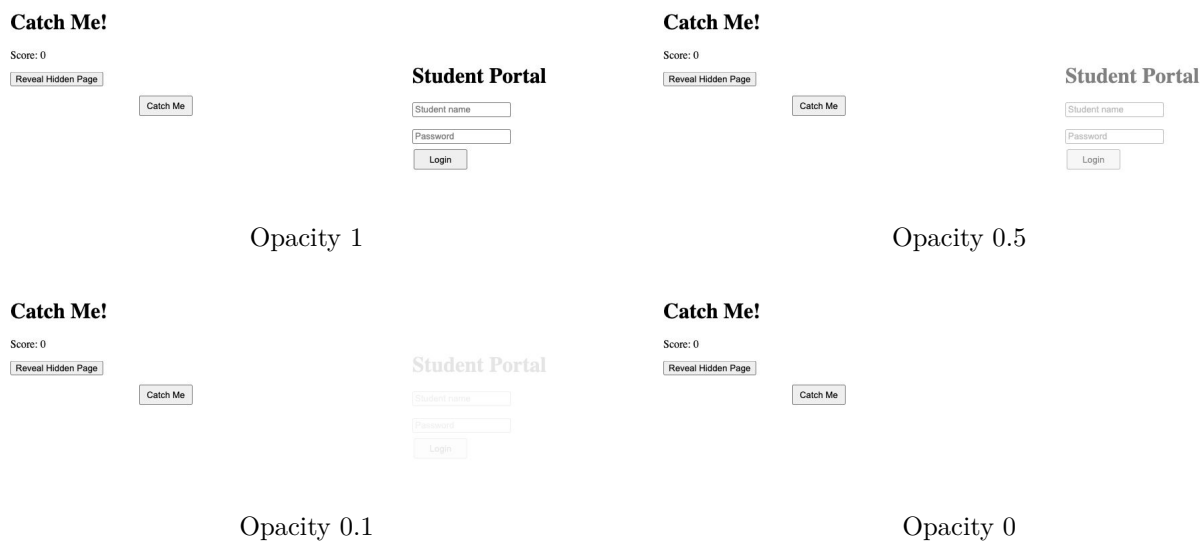


Figure 4: Effect of reducing iframe opacity.

6 Clickjacking moment

The attack is prepared when the score reaches 4. The iframe is moved to `left: 200px` and remains at `top: 80px`. Login is positioned 10 pixels from the iframe's left and 150 pixels from its top. Its position on the game page is therefore:

$$x = 200 + 10 = 210 \text{ px}, \quad y = 80 + 150 = 230 \text{ px}$$

The Catch Me button is placed at the same coordinates, (210px, 230px). The iframe has `z-index: 2`, while the game button has `z-index: 1`. Therefore, the next apparent game click is received by the hidden Login button.

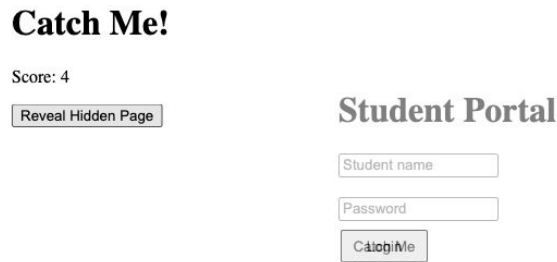


Figure 5: Partially visible iframe confirming the button overlap.

7 Reveal the attack

The Reveal Hidden Page button calls `revealAttack()`, which changes the iframe opacity to 0.67. This makes the portal partially visible and confirms that Login is positioned above Catch Me.

8 Anti-clickjacking protection

1. The main protections are the `X-Frame-Options` response header and the Content Security Policy `frame-ancestors` directive. A `SameSite` session cookie is an additional protection.
2. `X-Frame-Options: DENY` blocks all framing. `SAMEORIGIN` allows framing only by the same origin. CSP `frame-ancestors` can block all framing or allow a precise list of trusted origins.
3. Same-origin framing is allowed with either of these headers:

```
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: frame-ancestors 'self'
```

4. Because the Student Portal should never be framed, an appropriate configuration is:

```
Content-Security-Policy: frame-ancestors 'none'
X-Frame-Options: DENY
```

5. If another trusted university application must embed the portal, CSP should allow that exact origin:

```
Content-Security-Policy: frame-ancestors 'self'
https://trusted-app.university.example
```

Personal Research

CSP (and CORS) are two crucial mechanisms of web security when it comes to client-side, even though they can be very restrictive in some cases, attackers find new ways of gadgeting CSP through CSP-gadgets, especially when it comes to CSP that allows google assets, it is such a nice trick, I have used it very recently in a real-world XSS attack.

9 Conclusion

The experiment showed that opacity controls visibility but does not disable interaction. Clickjacking is prevented by restricting which origins may frame the protected page.

A Source code

A.1 hello.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Website</title>
</head>
<body>
  <h1>Welcome to My First Website</h1>
  <p>This is my first HTML page.</p>
  <button onclick="showMessage()">Click Me</button>

  <script>
    function showMessage() {
      alert("Hello from my first website!");
    }
  </script>
</body>
</html>
```

A.2 portal.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Portal</title>
  <style>
    #loginButton {
      position: absolute;
      top: 150px;
      left: 10px;
      width: 80px;
      height: 30px;
    }
  </style>
</head>
<body>
  <h1>Student Portal</h1>
  <input type="text" placeholder="Student name"><br><br>
  <input type="password" placeholder="Password">
  <button id="loginButton" onclick="login()">Login</button>

  <script>
    function login() {
      alert("yoo , ur in bro ");
    }
  </script>
</body>
</html>
```

A.3 game.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Catch Me!</title>
  <style>
    #catchButton {
      position: absolute;
      top: 150px;
      left: 200px;
      z-index: 1;
      width: 80px;
      height: 30px;
    }

    iframe {
      position: absolute;
      top: 80px;
      left: 670px;
      width: 300px;
      height: 220px;
      z-index: 2;
      opacity: 0;
      border: 0;
    }
  </style>
</head>
<body>
  <h1>Catch Me!</h1>
  <p>Score: <span id="score">0</span></p>
  <button id="catchButton" onclick="catchMe()">Catch Me</button>
  <iframe src="portal.html"></iframe>
  <button onclick="revealAttack()">Reveal Hidden Page</button>

  <script>
    let score = 0;
    function catchMe() {
      score++;
      document.getElementById("score").textContent = score;

      let button = document.getElementById("catchButton");
      let iframe = document.querySelector("iframe");

      if (score === 4) {
        button.style.left = "210px";
        button.style.top = "230px";
        iframe.style.left = "200px";
      } else {
        button.style.left = Math.floor(Math.random() * 400) + "px";
        button.style.top = Math.floor(Math.random() * 300 + 100) + "px";
      }
    }

    function revealAttack() {
      document.querySelector("iframe").style.opacity = "0.67";
    }
  </script>
</body>
</html>
```

References

- W3C, Content Security Policy Level 3
- MDN, Clickjacking
- OWASP, Clickjacking Defense Cheat Sheet